

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

**Ứng dụng DEX aggregator sử dụng kiến trúc intent
solver**

VŨ TIẾN AN NGUYỄN

Nguyen.VTA225148@sis.hust.edu.vn

Chương trình đào tạo: Kỹ thuật máy tính

Giảng viên hướng dẫn: TS. _____

Khoa: Kỹ thuật máy tính

Trường: Công nghệ thông tin và Truyền thông

HÀ NỘI, 06/2026

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

Ứng dụng DEX aggregator sử dụng kiến trúc intent solver

VŨ TIẾN AN NGUYỄN

Nguyen.VTA225148@sis.hust.edu.vn

Chương trình đào tạo: Kỹ thuật máy tính

Giảng viên hướng dẫn: TS. _____

Chữ kí GVHD

Khoa: Kỹ thuật máy tính

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 06/2026

LỜI CẢM ƠN

Trước hết, tôi xin gửi lời cảm ơn chân thành đến giảng viên hướng dẫn, người đã định hướng đề tài, góp ý về cách tiếp cận bài toán và chỉ ra những thiếu sót trong quá trình tôi xây dựng hệ thống. Những trao đổi trong các buổi gặp đã giúp tôi nhìn nhận lại nhiều quyết định thiết kế mà nếu chỉ tự làm một mình, tôi khó có thể nhận ra.

Tôi cũng xin cảm ơn các thầy cô trong Khoa Kỹ thuật Máy tính đã truyền đạt những kiến thức nền tảng trong suốt quá trình học tập tại trường, đó là cơ sở để tôi có thể tiếp cận và hoàn thành đồ án này.

Cuối cùng, tôi xin cảm ơn gia đình và bạn bè đã luôn động viên, tạo điều kiện để tôi tập trung hoàn thành đồ án tốt nghiệp trong khoảng thời gian vừa qua. Do thời gian và kiến thức còn hạn chế, đồ án chắc chắn không tránh khỏi thiếu sót, tôi rất mong nhận được sự góp ý từ thầy cô để hoàn thiện hơn.

TÓM TẮT NỘI DUNG ĐỒ ÁN

Các sàn giao dịch phi tập trung (DEX) hiện nay phần lớn vận hành dựa trên cơ chế tạo lập thị trường tự động (AMM), trong đó giá tài sản được xác định theo công thức toán học dựa trên lượng thanh khoản trong hồ chứa. Cách làm này khiến các giao dịch khối lượng lớn dễ gặp hiện tượng trượt giá và tạo cơ hội cho các tác nhân khai thác giá trị có thể trích xuất tối đa (MEV) thu lợi từ chính giao dịch đó. Một số giao thức đã chuyển sang mô hình giao dịch dựa trên ý định, trong đó người dùng ký lệnh và giao việc tìm đối tác, thực thi cho các bên thứ ba gọi là filler, tiêu biểu là UniswapX và CoW Protocol. Tuy nhiên, qua khảo sát có thể thấy phần lớn các giao thức này vẫn xử lý lệnh theo nguyên tắc tất cả hoặc không có gì, chưa ràng buộc trách nhiệm của filler một cách chặt chẽ, và hầu như chỉ hoạt động trong phạm vi một chuỗi.

Trước thực trạng đó, đồ án lựa chọn hướng tiếp cận giao dịch dựa trên ý định làm nền tảng, đồng thời bổ sung các cơ chế còn thiếu ở các hệ thống đã khảo sát, vì hướng đi này cho phép tách riêng việc thể hiện mong muốn giao dịch khỏi việc thực thi, qua đó giảm chi phí cho người dùng và tạo điều kiện tối ưu việc thực thi thông qua cạnh tranh giữa nhiều filler.

Trên cơ sở đó, đồ án xây dựng hệ thống NeutronX, gồm cơ chế khớp lệnh từng phần theo đấu giá kiểu Hà Lan kết hợp chuẩn ký dữ liệu EIP-712 và Permit2, cơ chế đặt cọc động ràng buộc trách nhiệm filler thông qua phạt, một thành phần thực thi dự phòng đảm bảo lệnh luôn được hoàn tất trước hạn, cùng một phần mở rộng cho phép giao dịch xuyên chuỗi giữa hai mạng tương thích EVM bằng hợp đồng khóa thời gian theo mã băm có hỗ trợ khớp từng phần thông qua cây Merkle.

Đóng góp chính của đồ án là đã thiết kế, cài đặt và kiểm thử thành công một hệ thống tổng hợp DEX hoàn chỉnh trên môi trường mạng giả lập, trong đó lệnh được khớp từng phần một cách minh bạch, filler chịu trách nhiệm về cam kết của mình, lệnh luôn được đảm bảo thực thi trước thời hạn, và người dùng có thể thực hiện giao dịch hoán đổi xuyên chuỗi mà không cần thông qua cầu nối trung gian.

Sinh viên thực hiện
(Ký và ghi rõ họ tên)

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	1
1.1 Đặt vấn đề.....	1
1.2 Mục tiêu, phạm vi và giới hạn.....	2
1.2.1 Mục tiêu.....	2
1.2.2 Phạm vi.....	2
1.2.3 Giới hạn.....	3
1.3 Định hướng giải pháp.....	3
1.4 Bố cục đồ án	3
CHƯƠNG 2. BỐI CẢNH VÀ ĐỀ XUẤT GIẢI PHÁP.....	5
2.1 Quá trình phát triển của Uniswap và chỗ còn thiếu.....	5
2.2 Tổng quan hệ thống NeutronX và mô hình lệnh	6
2.3 Khớp lệnh từng phần theo đấu giá kiểu Hà Lan	6
2.4 Đặt cọc và phạt.....	7
2.5 Cơ chế dự phòng.....	8
2.6 Giao dịch xuyên chuỗi không sử dụng cầu nối	9
2.7 Đánh đổi trong thiết kế.....	11
CHƯƠNG 3. THIẾT KẾ VÀ HIỆN THỰC HỢP ĐỒNG THÔNG MINH	12
3.1 Kiến trúc tổng thể và luồng giao dịch	12
3.2 Mô hình dữ liệu lệnh.....	13
3.3 Hợp đồng PartialFillReactor	14
3.4 Hợp đồng FillAuction	15
3.5 Hợp đồng FallbackExecutor.....	16
3.6 Các hợp đồng cho giao dịch xuyên chuỗi	17

3.7 Backend và Frontend.....	18
CHƯƠNG 4. KIỂM THỬ VÀ RÀ SOÁT BẢO MẬT	19
4.1 Phương pháp và môi trường kiểm thử	19
4.2 Kết quả kiểm thử chức năng.....	19
4.3 Kiểm thử đối kháng, bất biến và sinh ngẫu nhiên	21
4.4 Rà soát bảo mật	23
4.5 Kết luận chương	23
CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	25
5.1 Kết luận	25
5.2 Hạn chế	25
5.3 Hướng phát triển.....	26
TÀI LIỆU THAM KHẢO.....	29
PHỤ LỤC.....	31
A. ĐẶC TẢ USE CASE.....	31
A.1 Đặc tả use case “Thống kê tình hình mượn sách”	31
A.2 Đặc tả use case “Đăng ký làm thẻ mượn”	31

DANH MỤC HÌNH VẼ

Hình 2.1	Đường cong giá giảm dần theo block trong đầu giá kiểu Hà Lan và các điểm khớp lệnh từng phần.	7
Hình 2.2	Cách phân biệt hoàn cọc (filler không có lỗi) hay phạt cọc (filler bỏ dở) sau thời hạn.	8
Hình 2.3	Kiến trúc giao dịch xuyên chuỗi với một hợp đồng ký quỹ riêng cho mỗi slot trên cả hai chuỗi.	9
Hình 2.4	Cách tạo bí mật cho từng slot từ một bí mật gốc và cây Merkle tương ứng.	10
Hình 3.1	Kiến trúc tổng thể của hệ thống NeutronX gồm bốn tầng. . . .	12
Hình 3.2	Sơ đồ trình tự vòng đời một lệnh được khớp từng phần trong cùng một chuỗi.	13
Hình 3.3	Các trạng thái của một lệnh trong hợp đồng PartialFillReactor.	15
Hình 3.4	Một số màn hình của giao diện người dùng.	18
Hình 4.1	Kết quả chạy kịch bản kiểm thử đầu cuối cho giao dịch xuyên chuỗi.	21

DANH MỤC BẢNG BIỂU

Bảng 2.1	So sánh đặc tính giữa các giai đoạn của Uniswap và NeutronX.	6
Bảng 3.1	Các trường của một lệnh <code>PartialFillOrder</code> .	13
Bảng 3.2	Bảng hoàn cọc mặc định (đơn vị: phần vạn của số cọc), theo cỗ lệnh và tỉ lệ giao so với cam kết.	16
Bảng 3.3	So sánh hai cách làm xuyên chuỗi.	17
Bảng 4.1	Một số trường hợp kiểm thử quan trọng.	21
Bảng 5.1	Đối chiếu kết quả với các mục tiêu ban đầu.	25

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Thuật ngữ	Ý nghĩa
AMM	Cơ chế tạo lập thị trường tự động (Automated Market Maker)
API	Giao diện lập trình ứng dụng (Application Programming Interface)
DeFi	Tài chính phi tập trung (Decentralized Finance)
DEX	Sàn giao dịch phi tập trung (Decentralized Exchange)
EIP-712	Chuẩn ký dữ liệu có cấu trúc trên Ethereum (Ethereum Improvement Proposal 712)
ERC-20	Tiêu chuẩn token thay thế được trên Ethereum (Ethereum Request for Comment 20)
EVM	Máy ảo Ethereum (Ethereum Virtual Machine)
HTLC	Hợp đồng khóa thời gian theo mã băm (Hash Time Lock Contract)
MEV	Giá trị có thể trích xuất tối đa (Maximal Extractable Value)
RPC	Lời gọi thủ tục từ xa (Remote Procedure Call)

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Đặt vấn đề

Vài năm gần đây, tài chính phi tập trung (Decentralized Finance - DeFi) phát triển rất nhanh, và một trong những thành phần quan trọng nhất của nó là các sàn giao dịch phi tập trung (Decentralized Exchange - DEX). Khác với sàn giao dịch tập trung, DEX cho phép người dùng đổi tài sản số trực tiếp với nhau thông qua hợp đồng thông minh, mà không phải gửi tài sản cho một bên trung gian giữ hộ. Hiện nay khối lượng giao dịch mỗi ngày trên các DEX lớn như Uniswap đã lên tới hàng tỉ đô la, cho thấy mô hình này ngày càng được sử dụng nhiều.

Nếu nhìn lại quá trình phát triển của DEX, có thể thấy nó được cải tiến dần qua từng giai đoạn. Những DEX đầu tiên, tiêu biểu là Uniswap, hoạt động theo cơ chế tạo lập thị trường tự động (Automated Market Maker - AMM). Với cơ chế này, người dùng giao dịch trực tiếp với một hồ thanh khoản (liquidity pool), còn giá thì được tính bằng một công thức dựa trên lượng tài sản đang có trong hồ. Cách làm này đơn giản và lúc nào cũng có thể giao dịch được, nhưng nó có một điểm yếu: giá mà người dùng nhận được phụ thuộc hoàn toàn vào lượng thanh khoản đang có sẵn trong hồ trên chuỗi. Khi giao dịch một khối lượng lớn, giá sẽ bị lệch đi khá nhiều so với mong đợi (hiện tượng trượt giá), và điều này còn tạo cơ hội cho một số bên lợi dụng để kiếm lời từ chính giao dịch của người dùng (hiện tượng MEV, trình bày thêm ở Phụ lục ??).

Để khắc phục, gần đây xuất hiện một hướng đi mới gọi là giao dịch dựa trên ý định (intent-based). Ở hướng này, người dùng không tự thực hiện giao dịch nữa, mà chỉ ký một “ý định” mô tả mình muốn đổi gì lấy gì. Việc đi tìm nguồn thanh khoản và thực hiện giao dịch sẽ do các bên thứ ba (gọi là filler) cạnh tranh nhau làm. Điểm hay ở đây là filler không bị giới hạn trong một hồ thanh khoản nào cả; họ có thể gom thanh khoản từ nhiều nguồn khác nhau, kể cả những nguồn nằm ngoài chuỗi (off-chain). Nhờ vậy, người dùng có cơ hội nhận được giá tốt hơn so với khi chỉ giao dịch với một hồ AMM. Đây cũng chính là lý do đồ án chọn đi theo hướng này.

Tuy nhiên, khi tìm hiểu các hệ thống đang có (trình bày kỹ hơn ở Chương 2), có thể thấy vẫn còn ba vấn đề chưa được giải quyết tốt. Thứ nhất, hầu hết các hệ thống chỉ khớp lệnh theo kiểu được ăn cả ngã về không, tức là khớp toàn bộ lệnh hoặc không khớp gì, chứ chưa hỗ trợ khớp từng phần. Thứ hai, filler đăng ký nhận lệnh nhưng nếu sau đó họ không làm thì cũng không bị phạt gì, nên người dùng có thể phải chờ lâu hoặc lệnh bị bỏ dở cho tới khi hết hạn. Thứ ba, việc giao dịch giữa hai

chuỗi khác nhau (xuyên chuỗi) thường phải đi qua cầu nối (bridge), mà cầu nối lại là nơi giữ tài sản của người dùng nên dễ trở thành mục tiêu bị tấn công.

Từ những vấn đề trên, nếu xây dựng được một hệ thống cho phép khớp lệnh từng phần, bắt filler đặt cọc và chịu phạt nếu bỏ dở, đảm bảo lệnh luôn được hoàn tất trước khi hết hạn, đồng thời cho phép giao dịch xuyên chuỗi mà không cần đến cầu nối, thì người dùng sẽ có thêm một lựa chọn giao dịch tốt hơn và an toàn hơn. Đây chính là bài toán mà đề án hướng tới giải quyết.

1.2 Mục tiêu, phạm vi và giới hạn

1.2.1 Mục tiêu

Đề án đặt ra năm mục tiêu, tất cả đều nằm ở tầng hợp đồng thông minh - phần lõi của hệ thống:

- Xây dựng cơ chế khớp lệnh **từng phần** dựa trên **đấu giá kiểu Hà Lan**, trong đó giá giảm dần theo từng khối để hạn chế bị khai thác MEV.
- Cho phép người dùng ký lệnh ngoài chuỗi bằng chuẩn **EIP-712** kết hợp **Permit2**, nhờ đó không tốn phí gas khi tạo lệnh.
- Xây dựng cơ chế **đặt cọc và phạt** để ràng buộc trách nhiệm của filler khi nhận lệnh.
- Xây dựng một **cơ chế dự phòng** để chắc chắn phần lệnh còn lại luôn được khớp trước khi hết hạn.
- Hỗ trợ **giao dịch xuyên chuỗi** giữa hai mạng tương thích EVM mà không dùng cầu nối, đồng thời vẫn cho phép khớp từng phần.

1.2.2 Phạm vi

Đề án tập trung chủ yếu vào tầng **hợp đồng thông minh**, vì đây là nơi chứa toàn bộ logic quan trọng của hệ thống. Phần backend và frontend được làm ở mức vừa đủ để chạy thử và minh họa, chứ không phải là một sản phẩm hoàn chỉnh. Tài sản giao dịch giới hạn ở các token theo chuẩn ERC-20. Hệ thống được triển khai và kiểm thử trên mạng giả lập cục bộ tạo bằng Anvil: một mạng sao chép trạng thái từ Ethereum mainnet cho trường hợp giao dịch trong cùng một chuỗi, và hai mạng giả lập riêng cho trường hợp xuyên chuỗi.

Có một phần đề án quyết định không làm, đó là chiến thuật tính lợi nhuận của filler. Trong thực tế, filler có thể là các nhà giao dịch, nhóm nghiên cứu hoặc công ty hoạt động độc lập, và họ thường không công khai mã nguồn cũng như cách họ quyết định có tham gia khớp lệnh hay không. Vì vậy đề án chỉ tập trung xây dựng các quy tắc trên hợp đồng (đặt cọc, phạt, hoàn cọc) để ràng buộc filler, còn việc

filler làm thế nào để có lãi thì để cho họ tự lo. Hai filler đi kèm trong đề án chỉ là chương trình demo đơn giản dùng để chạy thử các luồng giao dịch, chứ không phải filler thật.

1.2.3 Giới hạn

Đề án mới chỉ chạy và kiểm thử trên mạng giả lập, chưa triển khai lên mạng chính thức (mainnet). Để chạy thật thì cần nhiều thứ mà đề án chưa có: thanh khoản thật, filler thật, một đợt kiểm toán bảo mật do người có chuyên môn thực hiện, cùng với chi phí gas đi kèm. Ngoài ra, một số chỗ trong thiết kế vẫn còn mang tính tập trung, ví dụ khóa đồng ký (cosigner) hiện do một bên duy nhất giữ. Các giới hạn này được nói rõ hơn ở Chương 5, kèm theo hướng khắc phục.

1.3 Định hướng giải pháp

Đề án chọn hướng giao dịch dựa trên ý định, trong đó người dùng (gọi là swapper) chỉ cần ký một lệnh nói rõ mình muốn giao dịch gì, còn việc tìm thanh khoản và thực hiện giao dịch thì để cho các filler cạnh tranh nhau làm. Hướng này tách riêng phần “nói ra ý muốn” khỏi phần “thực hiện”, nhờ đó người dùng đỡ tốn chi phí khi tạo lệnh, và việc thực hiện cũng có thể tốt hơn nhờ nhiều filler cạnh tranh.

Dựa trên hướng đó, đề án xây dựng một hệ thống đặt tên là **NeutronX**, gồm bốn thành phần làm việc cùng nhau: tầng hợp đồng thông minh lo việc kiểm tra và thực hiện lệnh trên chuỗi; tầng backend lo việc đồng ký lệnh và theo dõi trạng thái; tầng filler đóng vai các bên đi khớp lệnh; và giao diện người dùng để tạo và theo dõi lệnh. Kiến trúc chi tiết của hệ thống được trình bày ở mục 3.1.

Đóng góp chính của đề án gồm bốn cơ chế ở tầng hợp đồng: khớp lệnh từng phần theo dấu giá kiểu Hà Lan để hạn chế MEV; đặt cọc và phạt để ràng buộc filler; cơ chế dự phòng để lệnh luôn được hoàn tất; và phần mở rộng cho phép giao dịch xuyên chuỗi mà vẫn khớp được từng phần - điều mà qua tìm hiểu, đề án thấy chưa phổ biến ở các hệ thống xuyên chuỗi hiện nay. Cũng cần nói rõ rằng đây là một sản phẩm mang tính thử nghiệm, cải tiến một vài khía cạnh của hướng giao dịch theo ý định và được kiểm chứng trên môi trường giả lập, chứ không nhằm cạnh tranh với các hệ thống thương mại đã chạy nhiều năm.

1.4 Bố cục đề án

Phần còn lại của đề án được tổ chức như sau. Chương 2 tìm hiểu quá trình phát triển của DEX qua trường hợp Uniswap, chỉ ra chỗ còn thiếu mà đề án muốn bổ sung, rồi trình bày giải pháp NeutronX cùng bốn cơ chế đóng góp ở mức ý tưởng. Chương 3 đi vào thiết kế và cài đặt chi tiết tầng hợp đồng thông minh, và nói ngắn gọn về backend, frontend. Chương 4 trình bày việc kiểm thử và rà soát bảo mật của

hệ thống. Chương 5 tổng kết lại kết quả so với mục tiêu ban đầu, nêu các hạn chế còn lại và đề xuất hướng phát triển tiếp theo.

CHƯƠNG 2. BỐI CẢNH VÀ ĐỀ XUẤT GIẢI PHÁP

Chương này gồm hai phần. Phần đầu tìm hiểu hiện trạng giao dịch phi tập trung thông qua sự phát triển của Uniswap - sàn DEX lớn nhất hiện nay - để thấy rõ chỗ còn thiếu mà đề án muốn bổ sung. Phần sau trình bày tổng quan hệ thống NeutronX cùng bốn cơ chế đóng góp ở mức ý tưởng, kèm lý do vì sao thiết kế như vậy; còn cách cài đặt cụ thể thì để dành cho Chương 3.

2.1 Quá trình phát triển của Uniswap và chỗ còn thiếu

Thay vì so sánh dàn trải nhiều hệ thống, đề án chọn tìm hiểu kỹ một trường hợp tiêu biểu là Uniswap, vì đây là sàn giao dịch phi tập trung có khối lượng lớn nhất, và bản thân nó cũng đã đi qua nhiều giai đoạn phản ánh đúng sự phát triển chung của lĩnh vực. Có thể chia quá trình này thành ba giai đoạn.

Giai đoạn 1 - Uniswap v2/v3 (AMM). Đây là mô hình tạo lập thị trường tự động: người dùng swap trực tiếp với một hồ thanh khoản trên chuỗi, giá được tính theo công thức dựa trên lượng tài sản trong hồ. Ưu điểm là luôn sẵn sàng và hoàn toàn phi tập trung. Nhược điểm thì như đã nói ở mục 1.1: giá người dùng nhận được bị giới hạn bởi thanh khoản có sẵn trên chuỗi, nên dễ bị trượt giá khi giao dịch lớn.

Giai đoạn 2 - UniswapX (giao dịch theo ý định). Để giải quyết vấn đề trên, Uniswap đưa ra UniswapX theo mô hình ý định: người dùng ký lệnh ngoài chuỗi, rồi các filler cạnh tranh nhau thực hiện. Vì filler có thể lấy thanh khoản từ nhiều nguồn, kể cả ngoài chuỗi, nên người dùng có thể nhận được giá tốt hơn. UniswapX cũng dùng đấu giá kiểu Hà Lan (giá giảm dần để filler vào khi thấy có lãi) và bước đầu hỗ trợ giao dịch xuyên chuỗi. Cần nói rõ là phần lớn ý tưởng nền tảng mà đề án dựa vào đã có từ giai đoạn này.

Giai đoạn 3 - chỗ đề án muốn bổ sung. Dù vậy, vẫn còn ba điểm mà các hệ thống theo ý định hiện nay (kể cả UniswapX) chưa làm tròn vẹn, và đó cũng là phần đề án muốn cải tiến:

- **Khớp lệnh từng phần với giá được ký:** cho phép một lệnh được nhiều filler khớp dần thành nhiều phần, mỗi phần tính giá theo đường cong đã được xác thực, thay vì chỉ khớp trọn vẹn hoặc bỏ.
- **Ràng buộc filler bằng kinh tế:** bắt filler đặt cọc khi nhận lệnh và bị phạt nếu bỏ dở phần đã nhận, thay vì cho filler tham gia mà không phải chịu trách nhiệm gì.
- **Xuyên chuỗi khớp từng phần, không cầu nối:** chia một lệnh xuyên chuỗi thành nhiều phần độc lập bằng hợp đồng khóa thời gian theo mã băm kết hợp

cây Merkle, thay vì khớp nguyên lệnh qua một cầu nối trung gian.

Bảng 2.1 tóm tắt sự khác nhau theo từng đặc tính. Bảng chỉ ghi nhận một đặc tính *có* hay *không*, không nhằm nói hệ thống nào tốt hơn; mục đích là cho thấy NeutronX bổ sung thêm vài đặc tính theo cách ít gặp, chứ không phải để thay thế các hệ thống đã có.

Bảng 2.1: So sánh đặc tính giữa các giai đoạn của Uniswap và NeutronX.

Đặc tính	Uniswap v3	UniswapX	NeutronX
Lấy thanh khoản ngoài chuỗi	Không	Có	Có
Đầu giá kiểu Hà Lan	Không	Có	Có
Khớp lệnh từng phần	Một phần	Một phần	Có
Đặt cọc và phạt filler	Không	Không	Có
Đảm bảo khớp trước hạn	Không	Không	Có
Xuyên chuỗi khớp từng phần	Không	Một phần	Có

2.2 Tổng quan hệ thống NeutronX và mô hình lệnh

Ý tưởng chính của NeutronX là tách riêng hai việc: việc người dùng “nói ra ý muốn” và việc filler “thực hiện”. Người dùng (swapper) chỉ ký một lệnh mô tả mình muốn giao dịch gì; còn các filler thì đọc lệnh, đặt cọc rồi thực hiện từng phần trên chuỗi, dưới sự ràng buộc của các hợp đồng thông minh.

Toàn bộ thông tin của một giao dịch được gói trong một cấu trúc lệnh duy nhất (gọi là `PartialFillOrder`), gồm các trường mô tả tài sản vào và ra, khối lượng, mức đầu ra tối thiểu mà người dùng chấp nhận, thời hạn, cùng các tham số của đường cong giá đầu giá Hà Lan. Lệnh được ký theo chuẩn EIP-712 nên người dùng không tốn phí gas khi tạo lệnh (chi tiết EIP-712 và Permit2 xem Phụ lục ??); danh sách đầy đủ các trường được trình bày ở mục 3.2.

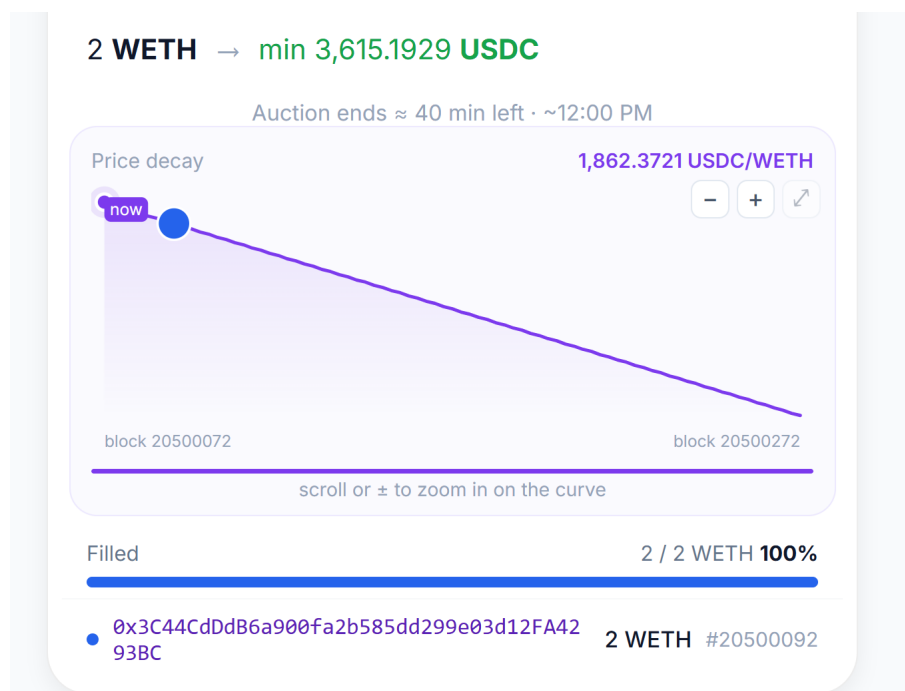
Bốn mục tiếp theo trình bày bốn cơ chế đóng góp, tương ứng với ba điểm còn thiếu đã nêu ở mục 2.1 cùng với cơ chế dự phòng.

2.3 Khớp lệnh từng phần theo đầu giá kiểu Hà Lan

Cơ chế đầu tiên cho phép một lệnh được khớp thành nhiều phần. Mỗi lệnh có kèm một đường cong giá theo kiểu đầu giá Hà Lan: giá bắt đầu ở mức cao, rồi giảm dần theo từng khối (block). Filler nào thấy mức giá hiện tại đủ để mình có lãi thì nhảy vào khớp một phần khối lượng; phần còn lại tiếp tục được chào ở giá thấp dần cho tới khi hết hoặc hết hạn. Nói cách khác, “đầu giá kiểu Hà Lan” ở đây chỉ là một cách để thị trường tự tìm ra mức giá mà tại đó có filler chịu vào (khái niệm đầy đủ xem Phụ lục ??).

Có hai điểm cần chú ý trong thiết kế. Một là, mỗi phần được tính giá độc lập tại

đúng mức giá đang có lúc khớp, để tránh việc tính dồn rồi bị tràn số ở phần cuối. Hai là, các tham số của đường cong giá được đưa vào phần dữ liệu ký, nên cũng được bên đồng ký (cosigner) xác thực; nếu không, một filler có thể tự đặt giá khởi điểm bằng một rồi chiếm đoạt tài sản của người dùng, nên bước xác thực này là bắt buộc.

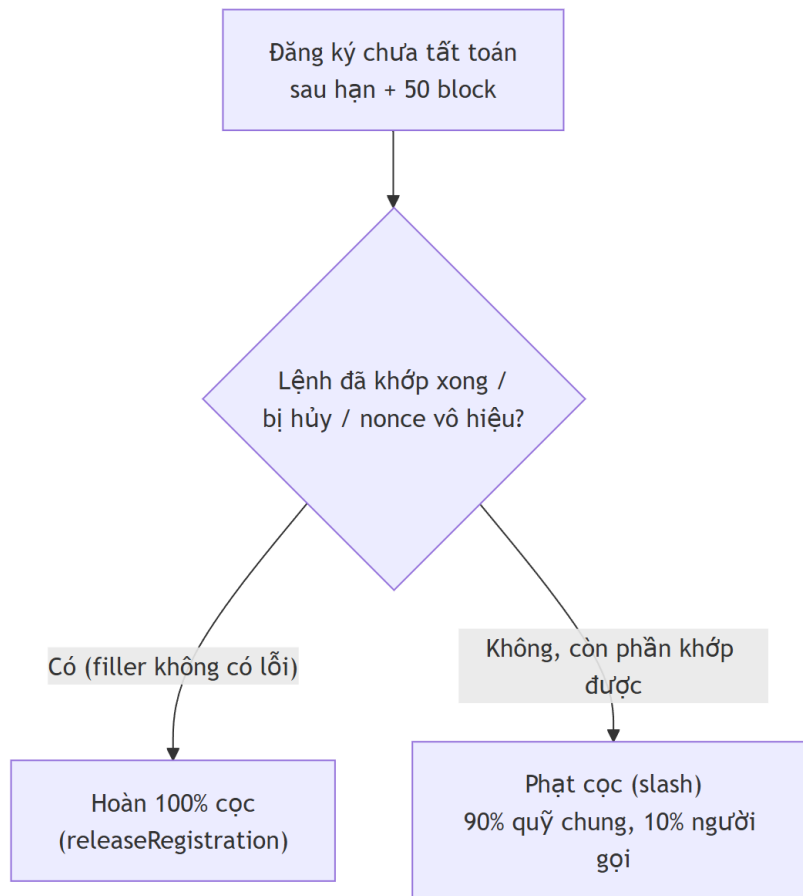


Hình 2.1: Đường cong giá giảm dần theo block trong đấu giá kiểu Hà Lan và các điểm khớp lệnh từng phần.

2.4 Đặt cọc và phạt

Cơ chế thứ hai - cũng là điểm khác biệt rõ nhất so với các hệ thống đã tìm hiểu - dùng kinh tế để ràng buộc filler. Khi đăng ký nhận một phần lệnh, filler phải bỏ ra một khoản cọc, và khoản cọc này được tính tùy theo cỡ lệnh và thời gian còn lại tới hạn. Nếu filler làm xong đúng phần đã nhận thì được lấy lại cọc; còn nếu chỉ giao thiếu so với phần mình đã cam kết thì phần thiếu sẽ bị giữ lại như một khoản phạt.

Điểm đáng chú ý của thiết kế là phân biệt được hai trường hợp dễ nhầm với nhau. Một filler “thua cuộc” (do filler khác đã khớp hết phần đó trước) thì không có lỗi và phải được hoàn cọc đầy đủ; chỉ filler thật sự “bỏ dở” - tức là bỏ lại phần còn khớp được mà mình đã nhận - mới bị phạt. Hình 2.2 minh họa cách phân biệt này. Nếu không có cơ chế cọc và phạt, một filler có thể đăng ký nhận lệnh rồi để đó, làm lệnh của người dùng treo cho đến khi hết hạn.



Hình 2.2: Cách phân biệt hoàn cọc (filler không có lỗi) hay phạt cọc (filler bỏ dở) sau thời hạn.

2.5 Cơ chế dự phòng

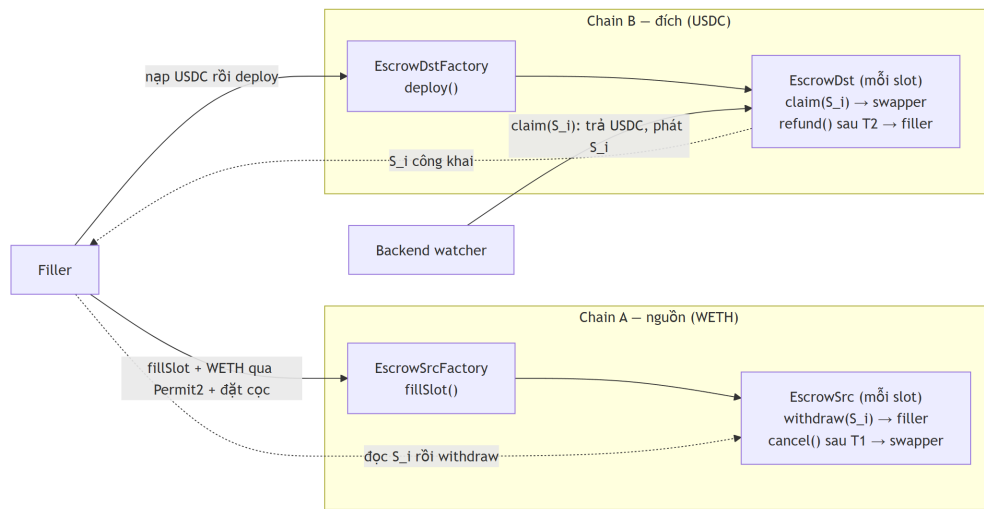
Cơ chế thứ ba giúp lệnh không bị bỏ dở khi hết hạn. Khi một lệnh đã gần tới hạn mà vẫn còn phần chưa được khớp, một thành phần dự phòng sẽ tự động hoán đổi nốt phần còn lại thông qua một sàn giao dịch (aggregator), với điều kiện kết quả nhận được vẫn phải đạt mức tối thiểu mà người dùng đã ký. Để tránh xung đột, luồng dự phòng và luồng khớp thông thường được thiết kế loại trừ lẫn nhau, không để cùng một phần khối lượng bị tiêu hai lần.

Về khả năng mở rộng, hợp đồng dự phòng có một danh sách trắng (allowlist) các sàn được phép gọi tới, nên không bị khóa cứng vào một sàn duy nhất. Tuy nhiên cũng cần nói thật là mỗi sàn mới cần thêm một bộ chuyển đổi (adapter) ở ngoài chuỗi để gọi API lấy đường đi và mã hóa dữ liệu giao dịch theo đúng định dạng riêng của nó; trong phạm vi đề án đã làm bộ chuyển đổi cho hai sàn là Uniswap và KyberSwap, và có thể thêm các sàn khác như 1inch hay 0x theo cùng cách. Sơ đồ chi tiết của luồng dự phòng được trình bày ở mục 3.5.

2.6 Giao dịch xuyên chuỗi không sử dụng cầu nối

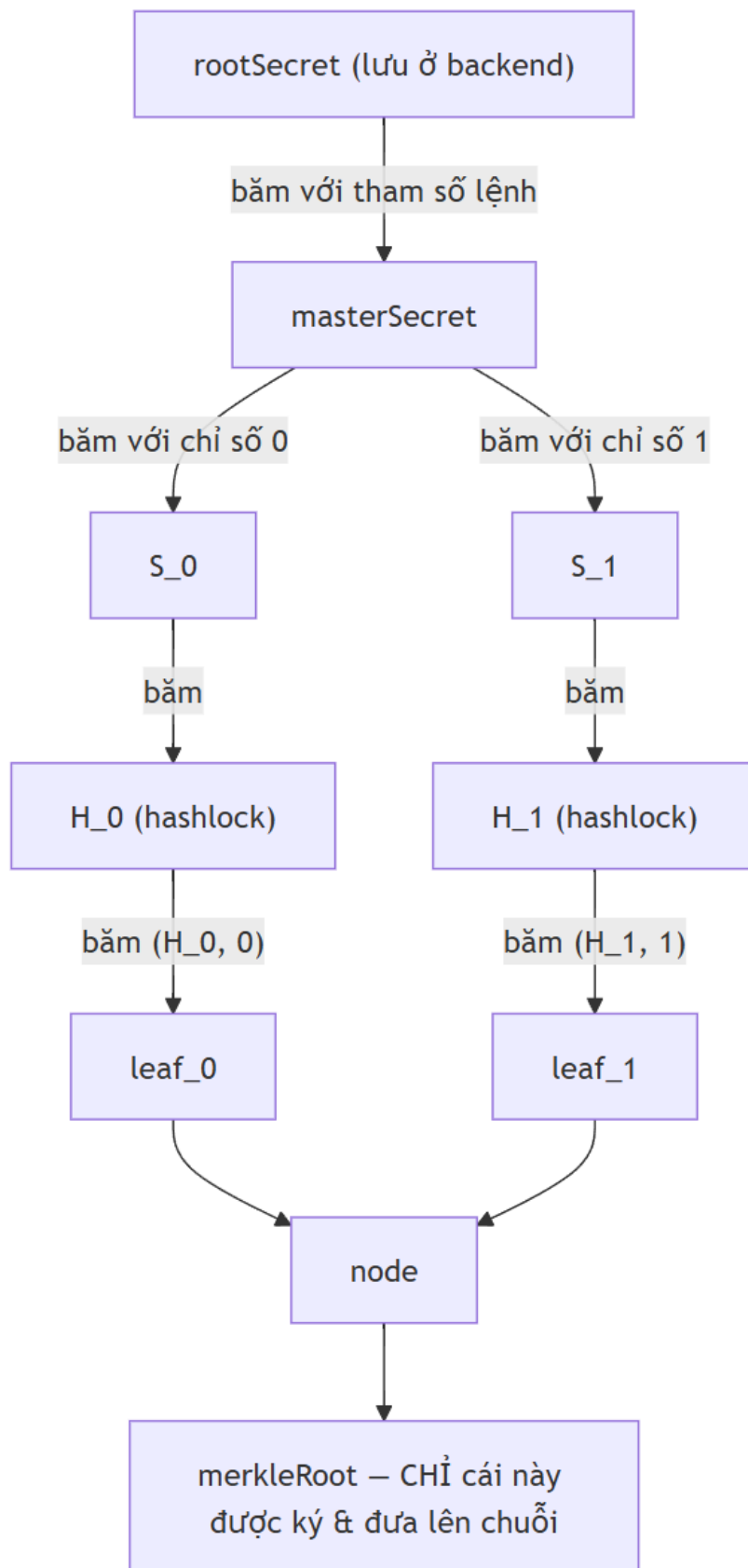
Cơ chế thứ tư mở rộng hệ thống sang giao dịch giữa hai chuỗi, và là phần ít gặp nhất. Các giải pháp xuyên chuỗi dựa trên hợp đồng khóa thời gian theo mã băm (Hashed Timelock Contract - HTLC) thường khớp nguyên lệnh theo kiểu được ăn cả ngã về không (khái niệm HTLC xem Phụ lục ??). NeutronX thì chia một lệnh xuyên chuỗi thành N phần (gọi là slot), và với mỗi slot, triển khai một hợp đồng ký quỹ *riêng* trên cả hai chuỗi. Nhờ vậy, nhiều filler có thể khớp nhiều phần một cách độc lập; một slot gặp sự cố cũng không ảnh hưởng tới các slot khác; và tài sản của những slot chưa được khớp thì không bao giờ rời khỏi ví người dùng.

Hình 2.3 mô tả kiến trúc trên hai chuỗi: ở mỗi chuỗi có một hợp đồng nhà máy (factory) lo việc tạo ra các hợp đồng ký quỹ cho từng slot; một thành phần theo dõi (watcher) ở backend lo việc tiết lộ bí mật để hoàn tất giao dịch. Một điều kiện về thời gian luôn được giữ để bảo vệ filler, đó là thời hạn trên chuỗi đích (T2) luôn sớm hơn thời hạn trên chuỗi nguồn (T1).



Hình 2.3: Kiến trúc giao dịch xuyên chuỗi với một hợp đồng ký quỹ riêng cho mỗi slot trên cả hai chuỗi.

Để khớp được từng phần mà chỉ phải ký một lần, hệ thống dùng một cây Merkle các bí mật. Từ một bí mật gốc, hệ thống tính ra bí mật riêng cho từng slot, rồi băm thành các khóa (hashlock) làm lá của cây Merkle; chỉ giá trị gốc (root) của cây được người dùng và cosigner ký và đưa lên chuỗi. Hình 2.4 minh họa cách sắp xếp này. Nhờ vậy, hệ thống cấp được quyền khớp cho từng slot một cách độc lập trong khi cả lệnh chỉ cần một chữ ký.



Hình 2.4: Cách tạo bí mật cho từng slot từ một bí mật gốc và cây Merkle tương ứng.

2.7 Đánh đổi trong thiết kế

Mỗi cơ chế ở trên đều có một sự đánh đổi mà đề án chấp nhận. Việc dùng một bên đồng ký (cosigner) giúp thiết kế đơn giản hơn, nhưng đổi lại tạo ra một điểm phụ thuộc tập trung. Việc tính cọc bằng cách quy giá trị lệnh về ETH dựa trên giá trung bình theo thời gian (TWAP) với cửa sổ ngắn thì hợp với môi trường giả lập, nhưng lại dễ bị thao túng hơn so với cửa sổ dài dùng trong thực tế. Cơ chế dự phòng thì cần thêm một bộ chuyển đổi cho mỗi sàn muốn tích hợp. Việc tính cọc bằng ETH cũng đòi hỏi token giao dịch phải có hồ thanh khoản ghép cặp với WETH. Những đánh đổi này, cùng các rủi ro còn lại, được nói rõ hơn ở chương kiểm thử và rà soát bảo mật (Chương 4) và khi tổng kết hạn chế ở Chương 5.

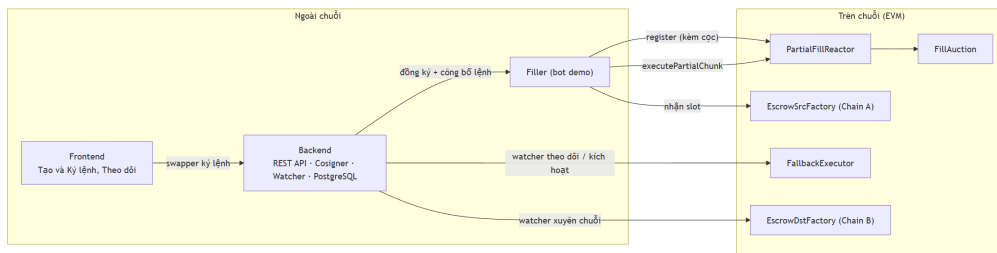
Tóm lại, chương này đã tìm hiểu quá trình phát triển của Uniswap để xác định chỗ còn thiếu, rồi trình bày bốn cơ chế đóng góp của NeutronX ở mức ý tưởng cùng lý do và sự đánh đổi của từng cơ chế. Chương tiếp theo sẽ đi vào thiết kế và cài đặt chi tiết các cơ chế này ở tầng hợp đồng thông minh.

CHƯƠNG 3. THIẾT KẾ VÀ HIỆN THỰC HỢP ĐỒNG THÔNG MINH

Chương này trình bày phần lõi của đề án: thiết kế và cài đặt các hợp đồng thông minh. Mở đầu là kiến trúc tổng thể và luồng đi của một giao dịch, sau đó đi vào từng hợp đồng theo đúng bốn cơ chế đã nêu ở Chương 2. Phần backend và frontend được nói ngắn gọn ở cuối chương vì chỉ đóng vai hỗ trợ.

3.1 Kiến trúc tổng thể và luồng giao dịch

Hệ thống NeutronX gồm bốn tầng. Tầng **hợp đồng thông minh** chạy trên chuỗi, chứa toàn bộ logic kiểm tra và thực hiện lệnh; đây là phần quan trọng nhất. Tầng **backend** chạy ngoài chuỗi, lo việc đồng ký lệnh, lưu trữ và theo dõi trạng thái. Tầng **filler** là các chương trình đi khớp lệnh. Cuối cùng là **giao diện người dùng** để tạo và theo dõi lệnh. Hình 3.1 mô tả cách bốn tầng này nối với nhau.



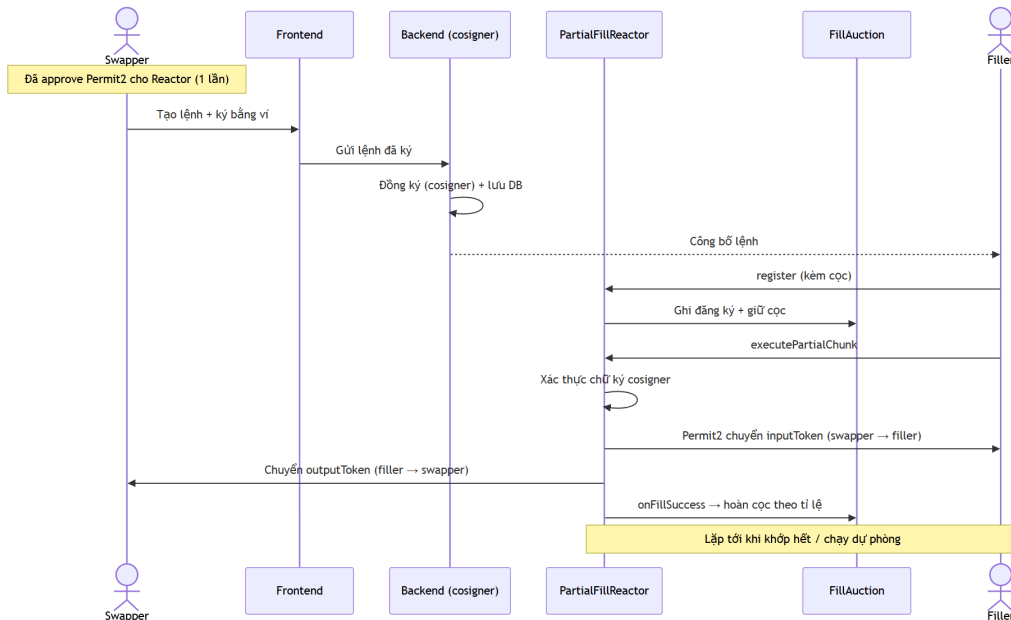
Hình 3.1: Kiến trúc tổng thể của hệ thống NeutronX gồm bốn tầng.

Để dễ hình dung, có thể theo dõi vòng đời của một lệnh giao dịch trong cùng một chuỗi qua các bước sau:

1. Người dùng tạo lệnh trên giao diện và ký bằng ví theo chuẩn EIP-712. Việc ký này diễn ra ngoài chuỗi nên không tốn phí gas.
2. Backend nhận lệnh, đồng ký (cosign) để xác nhận các tham số giá, rồi lưu vào cơ sở dữ liệu và công bố cho các filler.
3. Một filler thấy lệnh có lợi thì đăng ký nhận một phần khối lượng, kèm theo một khoản cọc, thông qua hợp đồng `FillAuction`.
4. Filler gọi hàm khớp lệnh trên `PartialFillReactor`. Hợp đồng dùng `Permit2` để chuyển tài sản vào của người dùng cho filler, đồng thời filler trả tài sản ra cho người dùng.
5. Sau mỗi lần khớp, `FillAuction` hoàn cọc cho filler theo mức họ đã giao đúng cam kết. Quá trình lặp lại cho tới khi lệnh được khớp hết.

6. Nếu gần tới hạn mà lệnh vẫn còn dư, `FallbackExecutor` sẽ khớp nốt phần còn lại.

Hình 3.2 biểu diễn luồng này dưới dạng sơ đồ trình tự.



Hình 3.2: Sơ đồ trình tự vòng đời một lệnh được khớp từng phần trong cùng một chuỗi.

3.2 Mô hình dữ liệu lệnh

Mọi thông tin của một lệnh được gói trong một cấu trúc gọi là `PartialFillOrder`, gồm 11 trường như ở Bảng 3.1. Toàn bộ 11 trường này đều nằm trong phần dữ liệu được ký, nên không bên nào có thể tự ý thay đổi mà không bị phát hiện.

Bảng 3.1: Các trường của một lệnh `PartialFillOrder`.

Trường	Ý nghĩa
<code>swapper</code>	Địa chỉ người tạo lệnh.
<code>inputToken</code>	Token người dùng bán ra.
<code>inputAmount</code>	Tổng khối lượng token bán.
<code>outputToken</code>	Token người dùng muốn nhận.
<code>minOutputAmount</code>	Mức nhận tối thiểu chấp nhận được (chống trượt giá).
<code>deadline</code>	Số block hạn chót của lệnh.
<code>nonce</code>	Số chống lặp lại lệnh.
<code>minFillBps</code>	Tỉ lệ tối thiểu cho mỗi lần khớp (tránh khớp quá vụn).
<code>startPrice</code>	Giá khởi điểm của đường cong đấu giá Hà Lan.
<code>decayPerBlock</code>	Mức giảm giá sau mỗi block.
<code>feeTier</code>	Mức phí pool dùng để tham chiếu giá.

Ba trường cuối (`startPrice`, `decayPerBlock`, `feeTier`) mô tả đường cong giá. Một điểm quan trọng trong thiết kế là ba trường này được đưa vào phần dữ liệu ký, nên đường cong giá mà filler thực hiện cũng được cosigner xác thực, filler không thể tự đặt giá có lợi cho mình.

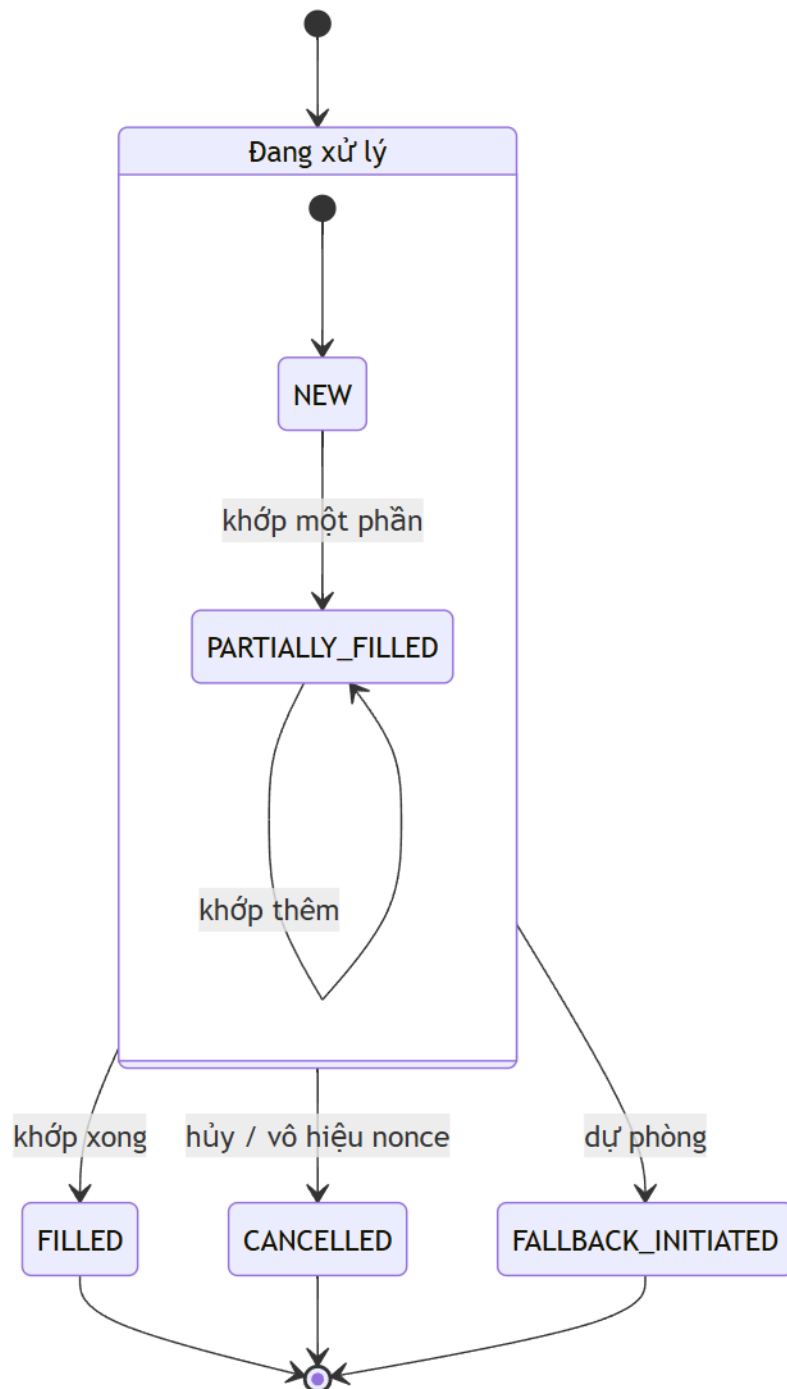
3.3 Hợp đồng `PartialFillReactor`

`PartialFillReactor` là hợp đồng lo việc khớp lệnh từng phần. Nó có hai hàm chính. Hàm `register` cho phép một filler đăng ký nhận một phần lệnh; hàm này chuyển tiếp yêu cầu đặt cọc sang `FillAuction` và đồng thời kiểm tra chữ ký của lệnh. Hàm `executePartialChunk` thực hiện một lần khớp.

Hàm khớp lệnh được viết theo trình tự “kiểm tra - thay đổi trạng thái - tương tác” (Checks-Effects-Interactions), một quy ước phổ biến để tránh lỗi gọi lại (reentrancy). Cụ thể, trước khi chuyển tài sản, hợp đồng kiểm tra: lệnh chưa bị hủy, chưa bị chuyển sang chế độ dự phòng, nonce chưa bị vô hiệu, filler có đăng ký hợp lệ, và lượng khớp không vượt phần còn lại. Sau khi qua hết các kiểm tra, hợp đồng mới cập nhật phần còn lại rồi mới chuyển tài sản.

Về cách tính giá, mỗi lần khớp được tính độc lập ở đúng mức giá đang có tại block đó, theo công thức đơn giản: số tiền ra bằng lượng khớp nhân với giá hiện tại. Cách tính độc lập như vậy tránh được một lỗi mà thiết kế ban đầu gặp phải, đó là khi tính dồn cả lệnh thì lần khớp cuối có thể bị tràn số và làm kẹt lệnh. Ngoài ra, để bảo vệ người dùng, mỗi lần khớp đều phải đạt một mức ra tối thiểu tính theo tỉ lệ, và khi lệnh khớp xong thì tổng số tiền ra phải đạt `minOutputAmount` mà người dùng đã ký.

Một lệnh trong hợp đồng có thể ở các trạng thái như ở Hình 3.3: mới tạo, đang khớp dở, đã khớp xong, hoặc bị hủy / chuyển sang dự phòng. Người dùng cũng có thể chủ động hủy lệnh hoặc vô hiệu một nonce để dừng các lần khớp tiếp theo.



Hình 3.3: Các trạng thái của một lệnh trong hợp đồng PartialFillReactor.

3.4 Hợp đồng FillAuction

`FillAuction` lo phần đặt cọc và phạt. Khi một filler đăng ký, hợp đồng tính ra số cọc cần nộp và ghi lại đăng ký đó. Số cọc được tính bằng cách quy giá trị phần lệnh về ETH, rồi nhân với một tỉ lệ theo cỡ lệnh và một hệ số theo thời gian còn lại tới hạn (đăng ký càng sát hạn thì hệ số càng cao). Giá trị quy về ETH được

lấy từ giá trung bình theo thời gian (TWAP) của pool Uniswap tương ứng.

Khi filler khớp xong, hàm `onFillSuccess` hoàn cọc theo bảng hoàn cọc ở Bảng 3.2. Điểm cần nhấn mạnh là mức hoàn được tính theo tỉ lệ giữa lượng filler đã giao và lượng họ đã *cam kết* của riêng mình, chứ không phải so với cả lệnh. Nhờ vậy, một filler làm đúng phần đã nhận (dù phần đó lớn hay nhỏ) đều được hoàn lại toàn bộ cọc; chỉ khi giao thiếu so với cam kết thì phần thiếu mới bị giữ lại.

Bảng 3.2: Bảng hoàn cọc mặc định (đơn vị: phần vạn của số cọc), theo cỡ lệnh và tỉ lệ giao so với cam kết.

Cỡ lệnh \ Tỉ lệ giao	0–2%	2–10%	10–30%	30–70%	≥70%
Nhỏ (< 1 ETH)	500	1000	2500	5000	10000
Vừa (1–10)	333	1000	2000	5000	10000
Lớn (10–100)	100	333	1000	3333	10000
Rất lớn (> 100)	100	200	667	2000	10000

Phần xử lý sau khi hết hạn được thiết kế để công bằng với filler. Sau thời hạn cộng thêm một khoảng chờ (50 block), nếu một đăng ký vẫn chưa tắt toán thì có hai khả năng. Nếu lệnh đã được khớp xong, đã bị hủy, hoặc nonce đã bị vô hiệu, thì filler không có lỗi và được hoàn lại toàn bộ cọc qua hàm `releaseRegistration`. Ngược lại, nếu lệnh vẫn còn phần khớp được mà filler bỏ dở, thì filler bị phạt qua hàm `slash`: phần lớn cọc (90%) vào quỹ chung, phần còn lại (10%) cho người gọi hàm. Cách phân biệt này đã được minh họa ở Hình 2.2 của Chương 2. Toàn bộ việc rút tiền dùng cơ chế “kéo” (pull payment): tiền được ghi nợ trước, người nhận tự gọi rút sau, để tránh lỗi khi chuyển tiền.

Có hai chi tiết thiết kế đáng chú ý. Thứ nhất, bảng hoàn cọc được chụp lại tại thời điểm đăng ký, nên dù chủ hợp đồng có sửa bảng sau đó cũng không ảnh hưởng tới các filler đã đăng ký trước. Thứ hai, số cọc tỉ lệ thẳng với cỡ phần đã nhận, nên không có chuyện đăng ký phần lớn lại rẻ hơn phần nhỏ.

3.5 Hợp đồng `FallbackExecutor`

`FallbackExecutor` lo cơ chế dự phòng. Nó chỉ được phép chạy trong khoảng 10 block trước hạn của lệnh. Khi chạy, hợp đồng kiểm tra chữ ký của lệnh, rồi đánh dấu lệnh sang chế độ dự phòng và đưa phần còn lại về 0 trong cùng một bước. Việc này khiến luồng dự phòng và luồng khớp thường loại trừ lẫn nhau, nên không có chuyện cùng một phần khối lượng bị tiêu hai lần.

Sau đó, hợp đồng dùng `Permit2` lấy phần còn lại của người dùng, rồi gọi tới một sàn giao dịch để hoán đổi. Kết quả nhận được phải đạt cả mức tối thiểu mà người dùng đã ký lẫn mức mà bên giải route tính sẵn; nếu thiếu thì cả giao dịch bị hủy.

Nếu route chỉ dùng hết một phần tài sản vào thì phần thừa được trả lại cho người dùng.

Về các sàn được hỗ trợ, hợp đồng giữ một danh sách trắng các địa chỉ sàn được phép gọi tới. Trong đồ án đã cài đặt bộ chuyển đổi cho **Uniswap** và **KyberSwap**: bộ chuyển đổi này nằm ở backend, lo việc gọi API của từng sàn để lấy đường đi tốt nhất rồi mã hóa dữ liệu giao dịch đúng định dạng của sàn đó. Muốn thêm một sàn mới chỉ cần viết thêm một bộ chuyển đổi tương tự và thêm địa chỉ sàn vào danh sách trắng, không phải sửa lại hợp đồng.

3.6 Các hợp đồng cho giao dịch xuyên chuỗi

Phần xuyên chuỗi dùng một mô hình khác với hợp đồng khóa thời gian dùng chung thường thấy: mỗi phần (slot) của lệnh có một hợp đồng ký quỹ riêng trên cả hai chuỗi, được tạo ra bằng kỹ thuật CREATE2. Trên chuỗi nguồn (ví dụ chuỗi giữ WETH), một hợp đồng nhà máy tên `EscrowSrcFactory` lo việc tạo các hợp đồng ký quỹ `EscrowSrc`; trên chuỗi đích (ví dụ chuỗi giữ USDC) là `EscrowDstFactory` tạo ra các `EscrowDst`.

Khi một filler nhận một slot, hợp đồng nhà máy kiểm tra hai chữ ký (của người dùng và của cosigner) cùng với bằng chứng Merkle cho slot đó, rồi mới tạo hợp đồng ký quỹ và dùng `Permit2` chuyển đúng phần tài sản của slot vào hợp đồng vừa tạo. Filler cũng phải nộp một khoản đặt cọc an toàn nhỏ kèm theo, để việc cố tình chiếm chỗ một slot rồi bỏ đó trở nên tốn kém. Cây Merkle các bí mật (đã trình bày ở mục 2.6) cho phép cấp quyền khớp từng slot một cách độc lập trong khi cả lệnh chỉ cần ký một lần.

Bảng 3.3 so sánh mô hình này với cách dùng một hợp đồng khóa thời gian dùng chung.

Bảng 3.3: So sánh hai cách làm xuyên chuỗi.

Tiêu chí	Hợp đồng dùng chung	Hợp đồng riêng mỗi slot
Cô lập sự cố	Một lỗi ảnh hưởng mọi filler	Một hợp đồng hỏng không ảnh hưởng slot khác
Tài sản của slot chưa khớp	Đã bị kéo vào hợp đồng	Vẫn nằm trong ví người dùng
Đặt cọc an toàn của filler	Không có	Có, mỗi slot một khoản
Dễ kiểm tra trạng thái	Khó	Mỗi hợp đồng tự báo trạng thái riêng

Một điều kiện thời gian luôn được giữ để bảo vệ filler: hạn trên chuỗi đích (T2) luôn sớm hơn hạn trên chuỗi nguồn (T1). Nhờ vậy, một filler đã bỏ tiền ra trên chuỗi đích nhưng vì lý do nào đó không hoàn tất được thì vẫn lấy lại được tiền của mình trước khi phần tài sản trên chuỗi nguồn bị hoàn cho người dùng.

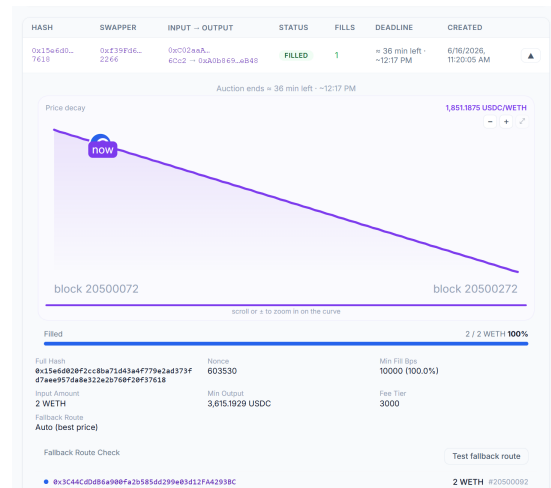
3.7 Backend và Frontend

Ngoài tầng hợp đồng, hệ thống còn có backend và frontend để kết nối người dùng với các hợp đồng. **Backend** viết bằng Node.js và TypeScript, dùng cơ sở dữ liệu PostgreSQL. Nó cung cấp các API để tạo và đọc lệnh, giữ vai trò *cosigner* (đồng ký để xác nhận đường cong giá và gốc cây Merkle), và chạy các thành phần theo dõi (watcher) để tự động kích hoạt cơ chế dự phòng cũng như hoàn tất giao dịch xuyên chuỗi. Một điều đáng chú ý là backend không hề giữ tài sản của người dùng: tài sản luôn nằm trong các hợp đồng trên chuỗi, nên kể cả khi backend gặp sự cố thì tiền của người dùng vẫn an toàn. **Frontend** là giao diện web để người dùng tạo lệnh (ký bằng ví), theo dõi tiến độ khớp và xem đường cong giá; một vài màn hình của nó được đưa ở Hình 3.4.

The screenshot shows a 'Swap' interface with the following details:

- You sell:** 2 WETH (Balance: 48 MAX)
- You receive (market estimate):** 3651.71 USDC (Balance: 103,724.7442)
- Auction range:** 3,724.7442 – 3,615.1929 USDC (min received)
- Price decay:** (USDC/WETH/block, max ~4.29e+3) 0.5326
- Fallback route:** Uniswap (Smart Order Router)
- Min fill (% of order per chunk):** 100
- Fillable by fillers:** 100%
- Min fill / chunk:** 100%
- Price (start – floor):** 1810.71 → 1757.45 USDC/WETH
- Swap button:** A large purple button at the bottom.

(a) Màn hình tạo lệnh.



(b) Màn hình theo dõi lệnh và đồ thị đấu giá.

Hình 3.4: Một số màn hình của giao diện người dùng.

Việc các hợp đồng và cơ chế vừa trình bày có hoạt động đúng như thiết kế hay không sẽ được kiểm chứng qua các thử nghiệm ở chương tiếp theo.

CHƯƠNG 4. KIỂM THỬ VÀ RÀ SOÁT BẢO MẬT

Sau khi đã thiết kế và cài đặt các hợp đồng ở chương trước, chương này kiểm tra xem chúng có thật sự chạy đúng và an toàn hay không. Phần đầu nói về cách kiểm thử và môi trường, phần giữa trình bày kết quả kiểm thử chức năng, và phần cuối là quá trình rà soát bảo mật.

4.1 Phương pháp và môi trường kiểm thử

Việc kiểm thử được làm ở hai mức. Mức thứ nhất là kiểm thử đơn vị (unit test) cho từng hợp đồng, viết bằng Foundry - một bộ công cụ phổ biến cho Solidity. Ngoài các test thông thường, đồ án còn dùng thêm kiểm thử bất biến (invariant test), tức là để công cụ tự sinh ngẫu nhiên rất nhiều chuỗi thao tác rồi kiểm tra một tính chất luôn phải đúng (ví dụ: tổng tiền trong hợp đồng không bao giờ bị âm). Mức thứ hai là kiểm thử đầu cuối (end-to-end), chạy cả một kịch bản giao dịch thật từ lúc tạo lệnh tới lúc tất toán.

Môi trường chạy là mạng giả lập cục bộ tạo bằng Anvil. Với kịch bản trong cùng một chuỗi, đồ án dùng một mạng sao chép trạng thái từ Ethereum mainnet, nhờ đó các token và pool Uniswap có sẵn để kiểm tra phần tính cọc theo giá thật. Với kịch bản xuyên chuỗi, đồ án dùng hai mạng Anvil riêng đóng vai hai chuỗi.

Ở đây cần nói rõ vài giả định của môi trường để hiểu đúng kết quả. Các filler trong kịch bản kiểm thử được cấp sẵn một lượng vốn lớn, và các block được tạo ra bằng lệnh thử công chứ không phải do người dùng thật giao dịch. Vì vậy, kết quả kiểm thử cho thấy các *luồng hoạt động* và *cơ chế phạt*, *hoàn cọc chạy đúng*, chứ chưa phản ánh được sự cạnh tranh kinh tế giữa các filler khi vốn có hạn và phí gas là thật.

4.2 Kết quả kiểm thử chức năng

Bộ kiểm thử có khoảng 126 trường hợp, trong đó 125 trường hợp đạt; một trường hợp không đạt là do giá trên mạng fork dao động chứ không phải do lỗi logic. Về độ phủ mã nguồn, hợp đồng `FillAuction` đạt 100% số dòng, còn `PartialFillReactor` đạt khoảng 94% số dòng. Phần dưới đây nói rõ hơn về các trường hợp kiểm thử quan trọng, nhóm theo bốn cơ chế chính, và Bảng 4.1 tóm tắt lại.

Khớp lệnh từng phần và giá. Nhóm này tập trung kiểm tra rằng người dùng không bị thiệt về giá. Một trường hợp quan trọng là khi filler cố tình sửa lại giá khởi điểm của lệnh thì giao dịch bị từ chối, vì giá đã được cosigner ký. Các trường hợp khác kiểm tra rằng một lần khớp trả ra ít hơn mức tối thiểu (cả khi khớp giữa chừng lẫn khi khớp nốt) đều bị chặn. Đáng chú ý, đồ án dùng một kiểm thử sinh

ngẫu nhiên (fuzz) rất nhiều lịch trình khớp khác nhau để chắc chắn rằng tổng số tiền người dùng nhận được không bao giờ thấp hơn mức họ đã ký, bất kể lệnh bị chia nhỏ thế nào. Ngoài ra còn có trường hợp chặn khớp quá vụn (nhỏ hơn `minFillBps`).

Đặt cọc và phạt. Nhóm này kiểm tra rằng việc phạt là công bằng. Trường hợp đáng chú ý nhất: khi hai filler tranh nhau một lệnh và một filler thua cuộc đua, filler thua phải lấy lại được toàn bộ cọc và *không* bị phạt; ngược lại, một filler nhận lệnh rồi bỏ dở thì bị phạt và không thu được lợi gì. Các trường hợp khác kiểm tra: khi người dùng hủy lệnh hoặc vô hiệu nonce thì filler đã đăng ký được hoàn cọc (vì lỗi không thuộc về họ); và một filler chỉ khớp một phần nhỏ nhưng đúng bằng phần đã cam kết thì vẫn được hoàn đủ cọc, tránh hiểu nhầm filler khớp phần nhỏ là kẻ xấu.

Cơ chế dự phòng. Trường hợp quan trọng nhất ở đây kiểm tra rằng sau khi cơ chế dự phòng đã chạy cho một lệnh thì không một filler nào còn khớp thường được lệnh đó nữa - đây chính là bảo đảm chống tiêu hai lần cùng một phần khối lượng. Các trường hợp khác kiểm tra: chặn chạy dự phòng quá sớm (chưa tới gần hạn), chặn gọi tới một sàn không nằm trong danh sách trắng, từ chối khi kết quả thấp hơn mức tối thiểu đã ký, và hoàn lại phần tài sản thừa cho người dùng nếu route không dùng hết.

An toàn chung. Đồ án còn có các test chuyên về an toàn, như chống tấn công gọi lại (reentrancy) và kiểm tra khả năng chi trả của hợp đồng. Vì cách viết hai test này có vài điểm đáng chú ý, chúng được trình bày riêng ở mục 4.3.

Bảng 4.1: Một số trường hợp kiểm thử quan trọng.

Nhóm	Trường hợp	Kết quả mong đợi	Đạt
Khớp từng phần	Filler sửa giá khởi điểm	Bị từ chối (giá đã được ký)	Có
	Khớp/hoàn tất dưới mức tối thiểu	Bị từ chối	Có
	Fuzz nhiều lịch trình khớp	Tổng ra luôn $\geq$ mức đã ký	Có
Cọc và phạt	Filler thua cuộc đua	Hoàn đủ cọc, không bị phạt	Có
	Filler nhận rồi bỏ dở	Bị phạt, không có lãi	Có
	Khớp phần nhỏ đúng cam kết	Hoàn đủ cọc	Có
Dự phòng	Khớp thường sau khi đã dự phòng	Bị chặn (chống tiêu hai lần)	Có
	Gọi sản ngoài danh sách trắng	Bị từ chối	Có
An toàn chung	Token độc hại gọi ngược (reentrancy)	Bị chặn	Có
Xuyên chuỗi	Hai filler khớp hai slot cùng lệnh	Cả hai nhận đúng tài sản	Có
	Bằng chứng Merkle / chữ ký sai	Bị từ chối	Có

Giao dịch xuyên chuỗi. Ngoài kịch bản chính, nhóm này kiểm tra nhiều tình huống biên: một slot không thể bị khớp hai lần; một bằng chứng Merkle sai hoặc chữ ký sai bị từ chối ngay trước khi đựng tới tài sản; khi hết hạn mà giao dịch chưa hoàn tất thì tài sản được hoàn cho người dùng kèm theo việc tịch thu khoản đặt cọc an toàn của filler; và điều kiện thời gian T2 sớm hơn T1 luôn được giữ để filler không bị mất tiền. Toàn bộ luồng còn được chạy đầu cuối với hai filler khớp hai slot khác nhau của cùng một lệnh: sau khi chạy hết các bước, kịch bản kiểm tra lại số dư của người dùng và hai filler trên cả hai chuỗi rồi báo “tất cả kiểm tra đều đạt”, như ở Hình 4.1.

Hình 4.1: Kết quả chạy kịch bản kiểm thử đầu cuối cho giao dịch xuyên chuỗi.

4.3 Kiểm thử đối kháng, bất biến và sinh ngẫu nhiên

Ngoài các trường hợp kiểm thử thông thường ở trên, đồ án còn dùng ba kiểu kiểm thử mạnh hơn để tăng độ tin cậy, vì với hợp đồng tài chính thì chỉ kiểm vài tình huống cụ thể là chưa đủ.

Kiểm thử sinh ngẫu nhiên (fuzz). Thay vì chỉ thử vài bộ số do người viết chọn, công cụ tự sinh ra rất nhiều bộ số khác nhau rồi kiểm tra một tính chất luôn phải đúng. Ví dụ rõ nhất là trường hợp đã nhắc ở trên: công cụ sinh ngẫu nhiên đủ kiểu

cách chia một lệnh thành nhiều phần (số phần, kích thước mỗi phần, thời điểm khớp) và kiểm tra rằng tổng số tiền người dùng nhận được khi lệnh khớp xong không bao giờ thấp hơn mức họ đã ký. Nhờ vậy, kết luận “người dùng không bị trả thiếu” không chỉ đúng cho vài ví dụ mà đúng cho rất nhiều khả năng.

Kiểm thử bất biến (invariant). Đây là kiểu kiểm thử trạng thái: công cụ tự sinh ra những chuỗi thao tác ngẫu nhiên gồm đăng ký, khớp, phạt, rút tiền và tua block, với người tham gia, số tiền và thời điểm bất kỳ; sau mỗi bước nó kiểm tra một tính chất quan trọng luôn phải giữ. Tính chất được chọn ở đây là *khả năng chi trả*: số ETH mà hợp đồng `FillAuction` đang giữ phải luôn bằng đúng tổng số cọc của các đăng ký chưa tắt toán cộng với tổng số tiền mọi người đang chờ rút. Điểm đáng nói nằm ở *cách* kiểm: bộ kiểm thử tự giữ một “sổ sách bóng” chạy song song với hợp đồng. Mỗi khi một đăng ký được tạo, sổ bóng cộng thêm số cọc; mỗi khi đăng ký được tắt toán hoặc bị phạt, sổ bóng trừ đi số cọc đó. Sau mỗi thao tác ngẫu nhiên, công cụ so số dư thật của hợp đồng với sổ bóng này - nếu lệch một đồng là phát hiện ngay:

```

1 // "so sach bong" chay song song voi hop dong:
2 function register(...) external { /* ... */ ghost_activeStake
   += stake; } // dang ky: + coc
3 function fill(...) external { /* ... */ ghost_activeStake
   -= stake; } // khop xong: - coc
4 function slash(...) external { /* ... */ ghost_activeStake
   -= stake; } // bi phat: - coc
5
6 // sau MOI thao tac ngau nhien, so that cua hop dong phai khop
   so bong:
7 function invariant_solveny() public view {
8     assertEq(address(auction).balance, ghost_activeStake +
        totalPending);
9 }

```

Kiểm thử đối kháng (adversarial). Nhóm này đóng vai một filler xấu (kiểu bot MEV) thắng cuộc đua sắp xếp giao dịch, rồi kiểm tra xem nó *lấy được* gì và *không lấy được* gì. Kết quả cho thấy bot xấu không thể sửa lại mức giá người dùng đã ký, và không thể trả cho người dùng thấp hơn mức tối thiểu. Một kết quả trung thực và đáng chú ý là: nếu bot cố tình khớp muộn để hưởng mức giá đã giảm thì nó vẫn “vượt” được một phần chênh lệch (trả người dùng ít hơn so với khớp sớm cùng khối lượng), nhưng phần chênh này luôn bị chặn dưới bởi sàn giá. Tức là hệ thống không loại bỏ hoàn toàn MEV, nhưng giới hạn được thiệt hại trong đúng khoảng mà người dùng đã chấp nhận khi ký lệnh. Kịch bản này được mô phỏng bằng cách cho filler xấu “thắng cuộc đua” - gọi thẳng các hàm theo đúng thứ tự nó muốn - rồi

đo xem nó rút được bao nhiêu.

Đáng chú ý nhất trong nhóm này là một test tự viết hằn một token ERC-20 “độc” để chứng minh hợp đồng chống được tấn công gọi lại (reentrancy). Ngay khi hợp đồng gọi token này để trả tiền cho người dùng giữa lúc tắt toán, token lập tức nhảy ngược trở lại hợp đồng để thử rút thêm một lần nữa:

```

1 // Token doc: dung luc duoc goi de tra tien, no nhay nguoc vao
   hop dong thu rut lai.
2 function transferFrom(address from, address to, uint256 amount)
3   public override returns (bool)
4   {
5     if (armed) {
6       armed = false;                                // chi
       re-enter dung mot lan
7       (bool ok, bytes memory ret) = target.call(
         attackCalldata);
8       if (!ok) { assembly { revert(add(ret, 0x20), mload(ret))
         } } // bong lai loi cua guard
9     }
10    return super.transferFrom(from, to, amount);
11  }

```

Cú gọi ngược này bị khóa chống reentrancy của hợp đồng chặn lại, làm cả giao dịch tắt toán thất bại đúng như mong muốn. Việc tự viết một token tấn công như vậy cho phép kiểm chứng phòng thủ trong điều kiện sát với một cuộc tấn công thật, thay vì chỉ kiểm trên giấy.

Đỉnh điểm của nhóm này là một kịch bản tổng hợp gồm nhiều lệnh và nhiều người tham gia, trộn lẫn filler trung thực và filler xấu: có lệnh được khớp sạch, có lệnh mà filler trung thực thua cuộc đua, có lệnh mà filler xấu đăng ký rồi bỏ và để filler trung thực khớp thay. Sau khi chạy xong, kịch bản kiểm tra một loạt tính chất toàn cục: mọi người dùng đều được khớp ở mức bằng hoặc trên sàn (không ai bị cướp); tổng token vào và ra khớp nhau; filler trung thực thua cuộc không mất đồng nào; filler hoàn thành đầy đủ thì được hoàn 100% cọc; và hợp đồng vẫn giữ đúng khả năng chi trả. Đây chính là bằng chứng mạnh nhất cho thấy thiết kế cọc và phạt hoạt động công bằng ngay cả khi có bên cố tình phá.

4.4 Rà soát bảo mật

(Phần này sẽ được bổ sung dựa trên kết quả rà soát bảo mật.)

4.5 Kết luận chương

Kết quả kiểm thử cho thấy các cơ chế chính của hệ thống hoạt động đúng như thiết kế trong môi trường giả lập: lệnh được khớp từng phần đúng giá, filler bị ràng

buộc bởi cọc và phạt một cách công bằng, lệnh luôn được hoàn tất nhờ cơ chế dự phòng, và giao dịch xuyên chuỗi diễn ra trơn vẹn với nhiều filler. Những hạn chế còn lại của hệ thống được bàn ở chương tiếp theo.

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết luận

Đồ án đã xây dựng được hệ thống NeutronX, một ứng dụng tổng hợp giao dịch phi tập trung theo kiến trúc intent solver, và kiểm chứng nó trên môi trường mạng giả lập. So với năm mục tiêu đặt ra ở mục 1.2.1, kết quả đạt được như Bảng 5.1.

Bảng 5.1: Đối chiếu kết quả với các mục tiêu ban đầu.

Mục tiêu	Kết quả
Khớp lệnh từng phần theo đầu giá Hà Lan	Đã cài đặt trong <code>PartialFillReactor</code> , mỗi phần tính giá độc lập, đã kiểm thử.
Ký lệnh ngoài chuỗi bằng EIP-712 và Permit2	Đã làm; người dùng ký bằng ví, không tốn phí gas khi tạo lệnh.
Đặt cọc và phạt ràng buộc filler	Đã cài đặt trong <code>FillAuction</code> , phân biệt được filler thua cuộc và filler bỏ dở.
Cơ chế dự phòng đảm bảo lệnh hoàn tất	Đã cài đặt trong <code>FallbackExecutor</code> , hỗ trợ Uniswap và KyberSwap.
Giao dịch xuyên chuỗi không cầu nối, khớp từng phần	Đã cài đặt theo mô hình ký quỹ riêng mỗi slot, chạy đầu cuối thành công với hai filler.

Như vậy, cả năm mục tiêu đều đã đạt được ở mức cài đặt và kiểm thử trên môi trường giả lập. Đóng góp đáng kể nhất của đồ án là cơ chế đặt cọc và phạt ràng buộc filler, và phần giao dịch xuyên chuỗi có hỗ trợ khớp từng phần - hai điểm mà các hệ thống đã khảo sát ít làm. Qua quá trình thực hiện, em cũng rút ra được nhiều kinh nghiệm về lập trình hợp đồng thông minh, đặc biệt là việc một sai sót nhỏ trong logic cũng có thể dẫn tới mất tài sản, nên việc kiểm thử kỹ và rà soát bảo mật là rất quan trọng.

5.2 Hạn chế

Bên cạnh các kết quả trên, đồ án vẫn còn một số hạn chế. Thứ nhất, hệ thống mới chỉ chạy trên mạng giả lập, chưa được triển khai lên mạng chính thức. Thứ hai, các filler trong kiểm thử được cấp vốn lớn nên đồ án mới kiểm chứng được tính đúng của các luồng, chưa mô phỏng được sự cạnh tranh kinh tế thật khi vốn có hạn; ngoài ra cũng chưa có thanh khoản và người dùng thật. Thứ ba, một số chỗ trong thiết kế còn mang tính tập trung, đáng kể nhất là khóa đồng ký (cosigner) hiện do một bên duy nhất giữ. Thứ tư, phần xuyên chuỗi mới làm cho hai chuỗi tương thích EVM, và cơ chế dự phòng mới hỗ trợ hai sàn là Uniswap và KyberSwap.

5.3 Hướng phát triển

Từ các hạn chế trên, đề án có thể được phát triển tiếp theo nhiều hướng. Trước hết là triển khai hệ thống lên mạng thử nghiệm rồi tiến tới mạng chính thức, kèm theo một đợt kiểm toán bảo mật do chuyên gia thực hiện. Tiếp theo, có thể phân tán vai trò cosigner cho nhiều bên thay vì một khóa duy nhất, để giảm sự phụ thuộc tập trung. Phần tính cọc theo giá có thể được làm chặt hơn bằng cách kéo dài cửa sổ lấy giá trung bình và kết hợp thêm một nguồn giá độc lập. Cơ chế dự phòng có thể bổ sung thêm các bộ chuyển đổi cho những sàn khác như 1inch hay 0x. Cuối cùng, phần xuyên chuỗi có thể mở rộng ra nhiều hơn hai chuỗi.

MỘT SỐ LƯU Ý VỀ TÀI LIỆU THAM KHẢO

Lưu ý: Sinh viên không được đưa bài giảng/slide, các trang Wikipedia, hoặc các trang web thông thường làm tài liệu tham khảo.

Một trang web được phép dùng làm tài liệu tham khảo **chỉ khi** nó là công bố chính thống của cá nhân hoặc tổ chức nào đó. Ví dụ, trang web đặc tả ngôn ngữ XML của tổ chức W3C <https://www.w3.org/TR/2008/REC-xml-20081126/> là TLTK hợp lệ.

Có năm loại tài liệu tham khảo mà sinh viên phải tuân thủ đúng quy định về cách thức liệt kê thông tin như sau. Lưu ý: các phần văn bản trong cặp dấu < > dưới đây chỉ là hướng dẫn khai báo cho từng loại tài liệu tham khảo; sinh viên cần xóa các phần văn bản này trong ĐATN của mình.

<**Bài báo đăng trên tạp chí khoa học:** Tên tác giả, tên bài báo, tên tạp chí, volume, từ trang đến trang (nếu có), nhà xuất bản, năm xuất bản >

[1] E. H. Hovy, "Automated discourse generation using discourse structure relations," *Artificial intelligence*, vol. 63, no. 1-2, pp. 341–385, 1993

<**Sách:** Tên tác giả, tên sách, volume (nếu có), lần tái bản (nếu có), nhà xuất bản, năm xuất bản>

[2] L. L. Peterson and B. S. Davie, *Computer networks: a systems approach*. Elsevier, 2007.

[3] N. T. Hải, *Mạng máy tính và các hệ thống mở*. Nhà xuất bản giáo dục, 1999.

<**Tập san Báo cáo Hội nghị Khoa học:** Tên tác giả, tên báo cáo, tên hội nghị, ngày (nếu có), địa điểm hội nghị, năm xuất bản>

[4] M. Poesio and B. Di Eugenio, "Discourse structure and anaphoric accessibility," in *ESSLLI workshop on information structure, discourse structure and discourse semantics*, Copenhagen, Denmark, 2001, pp. 129–143.

<**Đồ án tốt nghiệp, Luận văn Thạc sĩ, Tiến sĩ:** Tên tác giả, tên đồ án/luận văn, loại đồ án/luận văn, tên trường, địa điểm, năm xuất bản>

[5] A. Knott, "A data-driven methodology for motivating a set of coherence relations," Ph.D. dissertation, The University of Edinburgh, UK, 1996.

<**Tài liệu tham khảo từ Internet:** Tên tác giả (nếu có), tựa đề, cơ quan (nếu có), địa chỉ trang web, thời gian lần cuối truy cập trang web>

[6] T. Berners-Lee, *Hypertext transfer protocol (HTTP)*. [Online]. Available:

`ftp://info.cern.ch/pub/www/doc/http-spec.txt.Z` (visited on 09/30/2010).

[7] Princeton University, *Wordnet*. [Online]. Available: `http://www.cogsci.princeton.edu/~wn/index.shtml` (visited on 09/30/2010).

TÀI LIỆU THAM KHẢO

- [1] E. H. Hovy, “Automated discourse generation using discourse structure relations,” *Artificial intelligence*, vol. 63, no. 1-2, pp. 341–385, 1993.
- [2] L. L. Peterson and B. S. Davie, *Computer networks: a systems approach*. Elsevier, 2007.
- [3] N. T. Hải, *Mạng máy tính và các hệ thống mở*. Nhà xuất bản giáo dục, 1999.
- [4] M. Poesio and B. Di Eugenio, “Discourse structure and anaphoric accessibility,” in *ESSLLI workshop on information structure, discourse structure and discourse semantics, Copenhagen, Denmark*, 2001, pp. 129–143.
- [5] A. Knott, “A data-driven methodology for motivating a set of coherence relations,” Ph.D. dissertation, The University of Edinburgh, UK, 1996.
- [6] T. Berners-Lee, *Hypertext transfer protocol (HTTP)*. Accessed: Sep. 30, 2010. [Online]. Available: <ftp://info.cern.ch/pub/www/doc/http-spec.txt.Z>
- [7] Princeton University, *Wordnet*. Accessed: Sep. 30, 2010. [Online]. Available: <http://www.cogsci.princeton.edu/~wn/index.shtml>

PHỤ LỤC

A. ĐẶC TẢ USE CASE

Nếu trong nội dung chính không đủ không gian cho các use case khác (ngoài các use case nghiệp vụ chính) thì đặc tả thêm cho các use case đó ở đây.

A.1 Đặc tả use case “Thống kê tình hình mượn sách”

...

A.2 Đặc tả use case “Đăng ký làm thẻ mượn”

...